



SERVICIO DE ALMACENAMIENTO REMOTO Y LOCAL EN .NET

Nicolás Ortega Fernández y
Aitor Aróstegui Navarro
2º DAW

ÍNDICE

1. INTRODUCCIÓN

- 1.1. OBJETO DEL PROYECTO
- 1.2. ALCANCE
- 1.3. TECNOLOGÍAS UTILIZADAS

2. ANÁLISIS DE REQUISITOS

- 2.1. REQUISITOS FUNCIONALES
- 2.2. REQUISITOS NO FUNCIONALES
- 2.3. REQUISITOS DE INFORMACIÓN
- 2.4. REGLAS DE NEGOCIO

3. IMPLEMENTACIÓN TÉCNICA

4. CALIDAD Y PRUEBAS (QA)

- 4.1. PLAN DE PRUEBAS UNITARIAS
- 4.2. FLUJO DE TRABAJO Y CONTROL DE VERSIONES

5. ANÁLISIS ECONOMICO

1. INTRODUCCIÓN

1.1. OBJETO DEL PROYECTO

El objetivo principal de este proyecto es el diseño, desarrollo e implementación de un servicio backend web en **.NET** para la gestión asíncrona de usuarios. El sistema se fundamenta en una **arquitectura de almacenamiento jerárquico en tres niveles** (caché en memoria, base de datos local y API REST remota) para garantizar alto rendimiento, disponibilidad e integridad de datos. Además, integra mecanismos de sincronización en segundo plano y arquitectura reactiva basada en eventos.

1.2. ALCANCE

El alcance del proyecto contempla los siguientes componentes y funcionalidades:

- **Operaciones CRUD:** Implementación de un servicio Web API RESTful completo para la gestión de entidades de usuario.
- **Estrategia de Caché e Híbrida (Multi-Tier Storage):** Flujo de lectura optimizado en tres niveles (*MemoryCache / Redis → SQLite / PostgreSQL → API Remota JSONPlaceholder*) y persistencia en cascada.
- **Proceso de Sincronización Automática:** Desarrollo de un *BackgroundService* asíncrono para la limpieza y recarga periódica de datos cada 60 segundos, así como el restablecimiento inicial de la base de datos al arrancar el servicio.
- **Sistema de Notificaciones Reactivas:** Emisión y suscripción de eventos (*Rx.NET / System.Reactive*) activados tras operaciones de escritura (creación, edición y borrado) con salida por consola.
- **Mantenimiento y Calidad de Software:** Manejo de errores funcional con el patrón `Result<T, DomainError>`, registro de logs mediante *Serilog*, documentación técnica con **XMLDoc** y suite de pruebas unitarias/integración con **NUnit**, **Moq**, **FluentAssertions** y **Testcontainers**.
- **Contenedorización y Entornos:** Despliegue mediante **Docker** y **Docker Compose**.

1.3. TECNOLOGÍAS UTILIZADAS

Categoría	Tecnología / Herramienta	Descripción y Uso en el Proyecto
Lenguaje y Framework	C# / .NET	Plataforma principal para el desarrollo del servicio backend Web API.
IDE de Desarrollo	JetBrains Rider	Entorno de desarrollo integrado para la codificación, depuración y gestión del proyecto.
Persistencia de Datos (ORM / Mapeo)	Entity Framework Core / Dapper	EF Core para el modelado relacional y migraciones; Dapper para consultas de alto rendimiento.
Bases de Datos Relacionales	SQLite / PostgreSQL	SQLite como motor ligero por defecto en desarrollo local y PostgreSQL para producción/entorno contenedorizado.
Capa de Caché	MemoryCache / Redis	MemoryCache como caché primaria de baja latencia; Redis para caché distribuida en entornos en red.
Cliente HTTP / Integración API	Refit	Cliente HTTP fuertemente tipado para el consumo declarativo de la API REST de JSONPlaceholder.

Categoría	Tecnología / Herramienta	Descripción y Uso en el Proyecto
Programación Reactiva	System.Reactive (Rx.NET)	Gestión desacoplada y asíncrona del sistema de eventos y notificaciones.
Gestión de Errores y Logs	CSharpFunctionalExtensions / Serilog	Uso del tipo Result para flujo de errores explícito y Serilog para trazabilidad estructurada con rotación de archivos.
Contenedorización	Docker / Docker Compose	Orquestación y empaquetado de servicios, bases de datos (PostgreSQL/Redis) y ejecución multi-etapa.
Testing e Integración	NUnit, Moq, FluentAssertions, Testcontainers	Cobertura de pruebas unitarias (patrón AAA) y pruebas de integración con contenedores efímeros reales.

2. ANÁLISIS DE REQUISITOS

2.1 REQUISITOS FUNCIONALES

- **RF-01: Obtención de todos los usuarios (GET /api/users):** Retorna la lista completa de usuarios desde la base de datos local. En caso de no existir registros, consulta la API REST remota, los almacena localmente y los devuelve.
- **RF-02: Consulta de usuario por ID (GET /api/users/{id}):** Recupera la información de un usuario evaluando secuencialmente: Caché → Base de Datos Local → API REST. Si se encuentra en un nivel inferior, poblará los niveles superiores.

- **RF-03: Creación de usuarios (POST /api/users):** Envía la petición a la API REST remota, obtiene el identificador asignado, persiste el registro en la base de datos local, actualiza la caché y emite una notificación de creación.
- **RF-04: Actualización de usuarios (PUT /api/users/{id}):** Modifica los datos del usuario en la API REST. Al confirmarse, actualiza la base de datos local, la caché y notifica el evento.
- **RF-05: Eliminación de usuarios (DELETE /api/users/{id}):** Elimina la entidad en la API REST remota, remueve el registro de la base de datos local y la caché, y emite una notificación de borrado.
- **RF-06: Exportación de datos (GET /api/users/export):** Obtiene la lista consolidada de usuarios, la serializa en formato JSON mediante System.Text.Json y genera un archivo físico en el sistema de archivos local devolviendo su ubicación.
- **RF-07: Sincronización automática periódica:** Un servicio en segundo plano limpia la caché y la base de datos local cada 60 segundos, recargando los datos actualizados desde la API REST.

2.2. REQUISITOS NO FUNCIONALES

- **RNF-01: Rendimiento y Baja Latencia:** Minimización del tiempo de respuesta mediante la consulta prioritaria a la caché en memoria (*MemoryCache/Redis*).
- **RNF-02: Programación Asíncrona:** Todas las operaciones de lectura/escritura e I/O deben ser completamente asíncronas utilizando *async/Task*.
- **RNF-03: Mantenibilidad y Calidad de Código:** Empleo de características modernas de C# (Primary Constructors), documentación completa con XMLDoc e inyección de dependencias.
- **RNF-04: Portabilidad y Despliegue:** Soporte para ejecución en contenedores con Docker y Docker Compose sin dependencias externas obligatorias para el entorno básico.
- **RNF-05: Trazabilidad y Monitorización:** Registro estructurado de eventos en consola y ficheros con rotación de logs mediante Serilog.

2.3. REQUISITOS DE INFORMACIÓN

El sistema gestiona la entidad principal **Usuario (User)**, estructurada con los siguientes atributos obligatorios:

- **ID (id):** Identificador único del usuario (Numérico entero).
- **Nombre (name):** Nombre completo del usuario (Cadena de texto).
- **Nombre de usuario (username):** Identificador de cuenta o alias del usuario (Cadena de texto).
- **Correo electrónico (email):** Dirección de correo electrónico válida (Cadena de texto con formato email).

2.4. REGLAS DE NEGOCIO

- **RN-01 (Carga e Inicialización al Arranque):** Al iniciar la aplicación, la base de datos local se borra por completo y se repuebla automáticamente con los datos más recientes de la API REST remota.

- **RN-02 (Sincronización Periódica):** Cada 60 segundos, el *BackgroundService* debe invalidar toda la caché, vaciar la base de datos local y sincronizar la totalidad de los datos desde la API REST.
- **RN-03 (Notificaciones Asíncronas):** Toda operación de escritura exitosa (POST, PUT, DELETE) debe desencadenar obligatoriamente una notificación a través del servicio reactivo *System.Reactive* consumida en el arranque por *Program.cs*.
- **RN-04 (Manejo de Errores Funcional):** Las excepciones no controladas deben evitarse en favor de resultados explícitos *Result<T, DomainError>*. Si una entidad solicitada no existe en ninguna de las 3 capas, el sistema devolverá un error equivalente a 404 Not Found.

3. IMPLEMENTACIÓN TÉCNICA

- La aplicación ha sido desarrollada utilizando una arquitectura en capas basada en los principios **SOLID**, garantizando un alto grado de desacoplamiento, testabilidad y mantenibilidad. A continuación, se detallan y justifican las principales decisiones tecnológicas, patrones de diseño y herramientas utilizadas:
- **ASP.NET Core Web API:** Se ha seleccionado ASP.NET Core como tecnología base para el servicio backend al tratarse de un entorno de alto rendimiento, asíncrono y nativo en .NET. Permite exponer los endpoints RESTful (GET, POST, PUT, DELETE) de manera clara y eficiente, integrando de forma transparente middleware de registro de logs, inyección de dependencias y tareas en segundo plano (*BackgroundServices*).
- **Arquitectura de Almacenamiento en 3 Niveles (Multi-Tier Storage):** Se implementó este patrón para optimizar el rendimiento de lectura y reducir la carga sobre la API externa. La estrategia consiste en consultar progresivamente:
- **Caché L1 (MemoryCache / Redis):** Respuesta de ultra-baja latencia para datos de acceso frecuente.
- **Persistencia L2 (EF Core / Dapper con SQLite / PostgreSQL):** Almacenamiento relacional local que garantiza disponibilidad y soporte offline.
- **Fuente de Verdad L3 (Refit / JSONPlaceholder):** Cliente HTTP fuertemente tipado para sincronizar con la API remota.
- **Patrón Result y ROP (Railway-Oriented Programming):** Se ha empleado la librería *CSharpFunctionalExtensions* con el tipo *Result<T, DomainError>* para la gestión de errores y validaciones en lugar de lanzar excepciones controladas. La programación orientada a vías (ROP) se ha preferido sobre el uso de excepciones debido a que estas últimas están diseñadas para situaciones verdaderamente anómalas e impredecibles

del sistema; un error de validación (como un email con formato incorrecto o la no existencia de un usuario) es una respuesta esperable dentro del flujo de negocio. El uso de Result hace que el flujo de control sea explícito, el código sea más limpio, mantenible y los errores altamente predecibles.

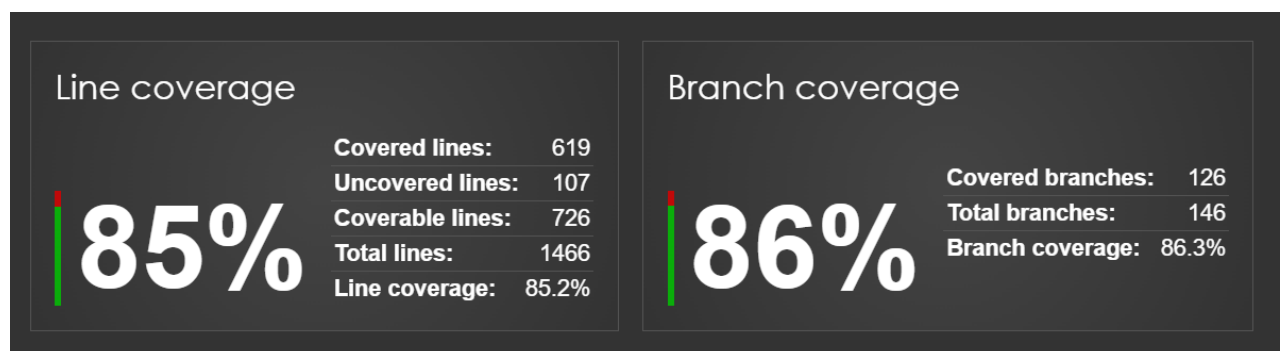
- **Programación Reactiva (Rx.NET / System.Reactive):** Se ha adoptado la programación reactiva para implementar el servicio de notificaciones desacoplado (`IObservable<T>`). Esto permite que el servicio `UserService` emita eventos de modificación de datos (creación, edición y borrado) de manera asíncrona sin conocer qué componentes reaccionarán a dichos eventos, facilitando la suscripción de múltiples manejadores (como logs por consola o futuros canales de notificación).
- **Inyección de Dependencias (DI):** Se ha utilizado el contenedor de dependencias nativo de .NET. Su implementación permite aplicar el principio de Inversión de Dependencias (DIP) y se ha justificado por dos motivos fundamentales:
- **Testabilidad:** Permite reemplazar repositorios, clientes HTTP y servicios de caché por *mocks* (*Moq*) en la suite de pruebas unitarias con *NUnit* y *FluentAssertions*.
- **Configurabilidad y Flexibilidad:** Facilita cambiar la configuración del motor de base de datos (SQLite o PostgreSQL) o del proveedor de caché (MemoryCache o Redis) mediante el archivo de configuración `appsettings.json` sin alterar la lógica de negocio subyacente.

4. CALIDAD Y PRUEBAS (QA)

4.1. PLAN DE PRUEBAS UNITARIAS

Se han llevado a cabo pruebas unitarias y pruebas de dobles con la finalidad de verificar y garantizar la calidad del código.

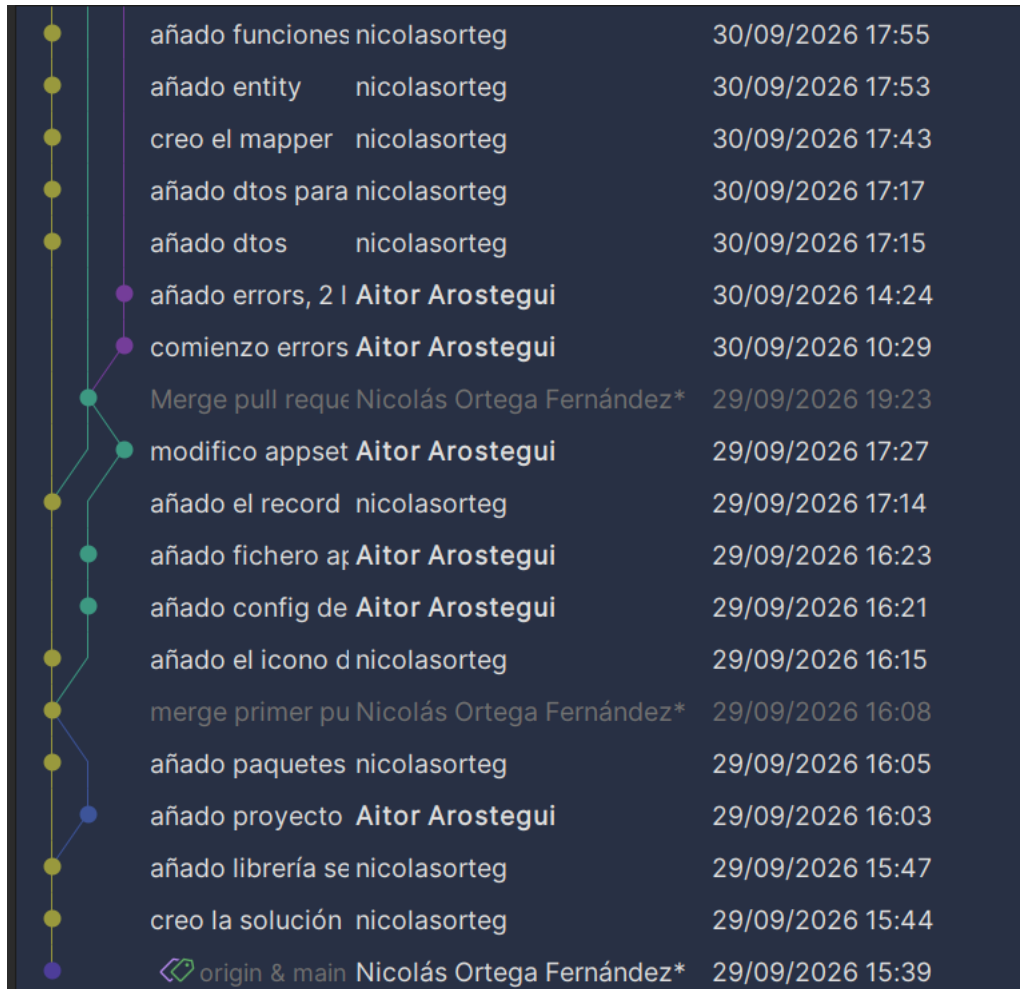
Estas pruebas han ayudado a verificar que el código programado hace lo esperado

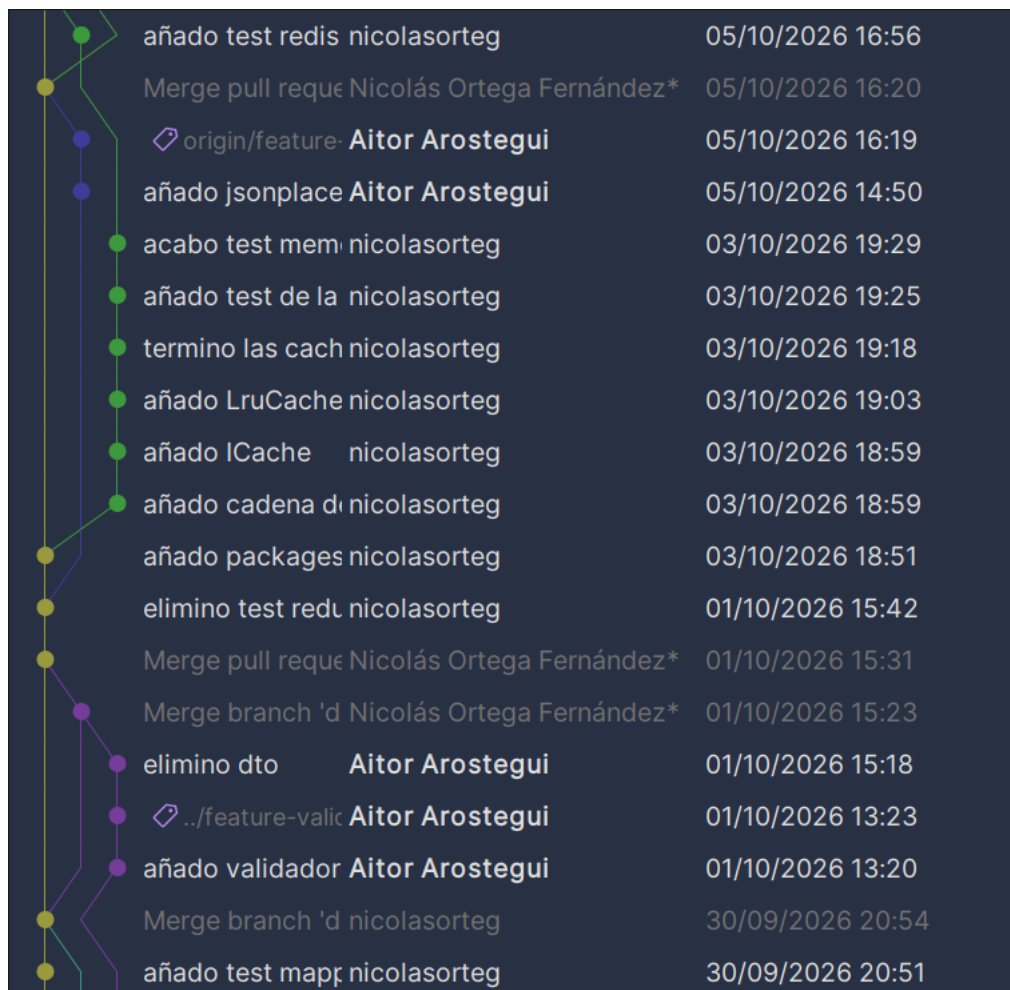


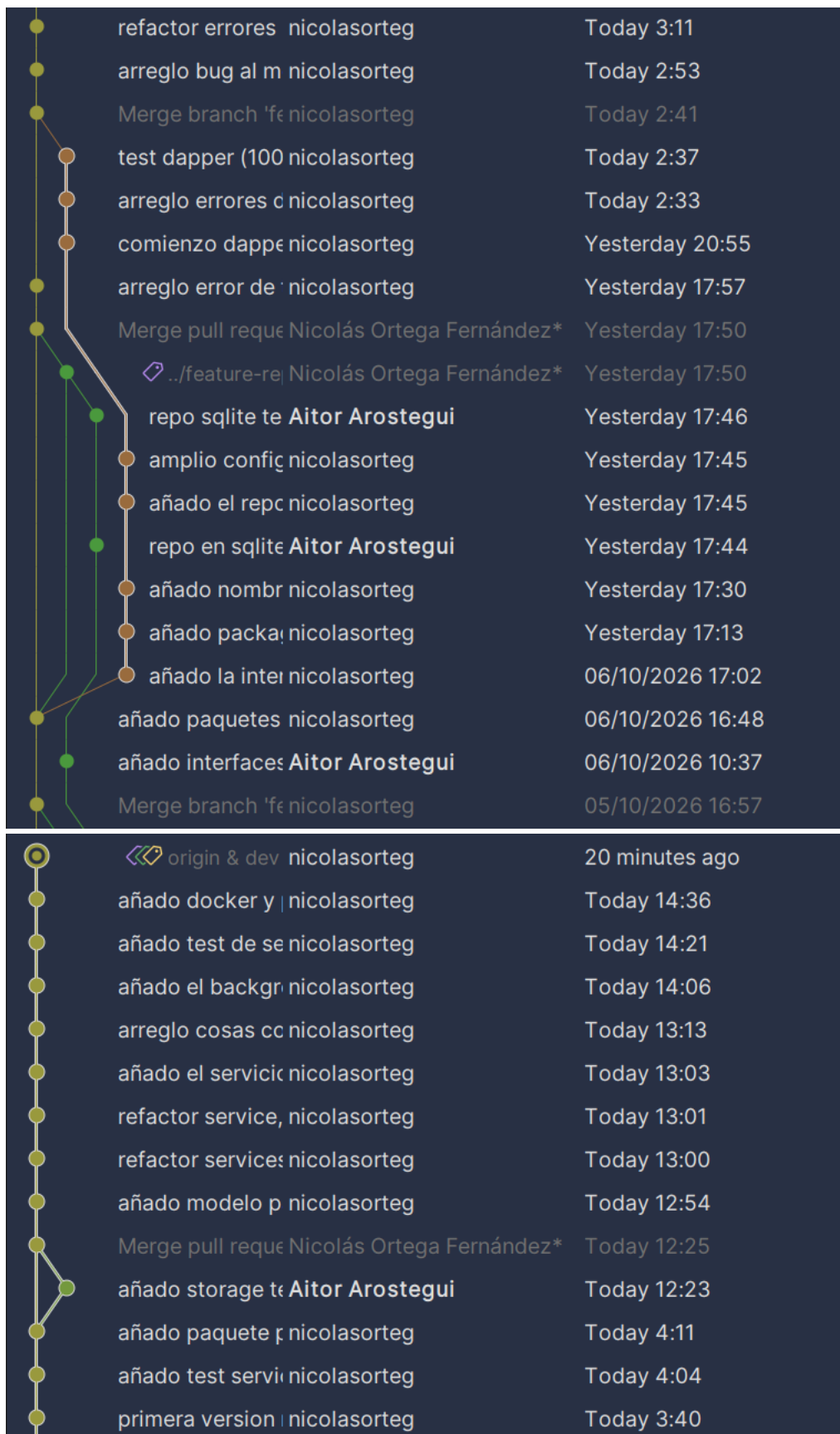
4.2. FLUJO DE TRABAJO Y CONTROL DE VERSIONES

Se ha usado Git con repositorio alojado en Github durante todo el desarrollo para tanto poder trabajar desde varios dispositivos como para llevar a cabo un control de los cambios en el proyecto y ver cómo ha ido evolucionando.

Se adjunta a continuacion el diagrama de GitFlow:







5. ANÁLISIS ECONÓMICO

La estimación de costes del proyecto se ha estructurado en tres bloques principales: coste de desarrollo y producción, coste de licencias y tecnologías, y coste de mantenimiento.

Coste de Desarrollo y Producción

El desarrollo del servicio de almacenamiento jerárquico y sincronización ha tenido una duración aproximada de **1,5 semanas (10 días laborables en total)**. Descontando 2 días de inactividad, se han completado **8 días de trabajo efectivo**.

El proyecto ha sido desarrollado en pareja por **2 desarrolladores**, dedicando un promedio conjunto de **5,5 horas diarias** (aproximadamente 2,75 horas al día por desarrollador). Esto suma un total de **44 horas de trabajo efectivo** (8 días $\times$ 5,5 h/día).

Tomando como referencia el coste medio por hora de un perfil de **Desarrollador .NET Junior** estimado en **14 €/h**, el coste total de producción asciende a **616,00 €** (44 h $\times$ 14 €/h).

Coste de Tecnologías y Herramientas

La gran mayoría del stack tecnológico empleado es de código abierto (*Open Source*) y no genera costes de licenciamiento (.NET 9, SQLite, PostgreSQL, Redis, Docker, Refit, NUnit).

Sin embargo, para un entorno de desarrollo profesional y comercial, se contempla la adquisición de licencias comerciales del entorno de desarrollo **JetBrains Rider**, cuyo coste es de aproximadamente 150 € anuales por desarrollador. Para un equipo de 2 desarrolladores, esto supone un coste de **300,00 €**. *(Nota: En el ámbito académico/educativo, estas licencias son provistas de forma gratuita por la institución).*

Coste de Mantenimiento

El mantenimiento preventivo, correctivo y de actualización del servicio se ha calculado aplicando un **20% anual** sobre la suma del coste de desarrollo y las licencias tecnológicas iniciales (que suman una base de 916,00 €). Esto resulta en un coste de mantenimiento de **183,20 € al año**.

Garantizar una cobertura y soporte integral del servicio durante un periodo de **2 años** supone un coste total de mantenimiento de **366,40 €** (183,20 € $\times$ 2).

Resumen del Coste Total

Sumando los tres conceptos evaluados, el coste global estimado para el desarrollo, despliegue y mantenimiento del proyecto durante 2 años se desglose de la siguiente manera:

- **Coste de Producción:** 616,00 € (44 horas de trabajo a 14 €/h)
- **Coste de Tecnologías:** 300,00 € (2 licencias comerciales de JetBrains Rider)
- **Coste de Mantenimiento:** 366,40 € (cobertura integral durante 2 años)

COSTE TOTAL ESTIMADO DEL PROYECTO: 1.282,40 €